6. Simulation: a virtual construction site in Blender

Why simulate

On a real site we never know the exact truth, so we cannot say exactly how wrong the model is. In a virtual site we know every wall to the millimetre and the true path of the scanner (ground truth). The simulation lets us test the **unchanged** live stack and the pipeline, and score them in centimetres.

What is simulated

Piece	Tool	File
Construction site, two days (day 2 has changes)	Blender 5.2	`sim/blender/gen_world.py`
The DT-1 handheld device with Airy or Mid-360 + ZED 2i	Blender	`sim/blender/gen_device.py` (writes `rig_sim.yaml`)
Operator walking with the device	Blender	`sim/blender/walk.py` (`trajectory.csv`, 1 kHz, first 3 s still)
Camera images	Blender EEVEE	`sim/blender/render_cams.py`
LiDAR and IMU	Open3D ray casting (Embree), ~3.5 M rays/s	`sim/dt_sim` (`lidar_airy.py`, `lidar_mid360.py`, `imu.py`)
Scoring against truth	Python	`dt_sim eval`
Result pictures	Blender	`sim/blender/show_result.py`

Why there is a file-based bridge

Blender 5.2 uses Python 3.13; ROS 2 Humble uses Python 3.10, so Blender cannot load ROS directly. We built two bridges:

- **Blender → ROS (sensor data):** Blender exports the world, path and device; `dt_sim` writes bags with the **same topics the real drivers publish** (`/rslidar_points` or `/livox/lidar`, raw IMU in g, ZED images); `ros2 bag play --clock` feeds them to the real stack.
- **ROS → Blender (live view):** `dt_blender_bridge` sends the estimated pose and growing map over TCP (port 5577); the **DT Live** add-on draws them in Blender.

The whole demo (`sim/run_demo.sh`)

```
flowchart TD
    subgraph BL["Blender"]
        W["gen_world.py<br/>day1 + day2"] --> DV["gen_device.py<br/>DT-1 +
rig_sim.yaml"]
        DV --> WK["walk.py<br/>operator path"]
        WK --> RC["render_cams.py<br/>ZED frames"]
    end
    subgraph SIMB["dt_sim"]
        BG["dt_sim bag<br/>LiDAR + IMU ray casting,<br/>driver-native MCAP
bags"]
    end
    subgraph STACK["Real ROS 2 stack, sim time"]
        PL["ros2 bag play --clock"] --> BRG["dt_lidar_bridge"]
        BRG --> FL["FAST-LIO"]
        FL --> RCD["record session bag"]
        FL -.-> LV["dt_blender_bridge<br/>DT Live view"]
    end
    subgraph PIPE["dtp + scoring"]
        TJ["traj + map + colorize"] --> AD["align + diff day1 vs day2"]
        AD --> EV["dt_sim eval<br/>ATE, map error, completeness"]
        EV --> SR["show_result.py<br/>result renders"]
    end
    RC --> BG
    WK --> BG
    BG --> PL
    RCD --> TJ
```

Run it:

```
cd ~/Music/flo-mobility/digital-twin
source scripts/jetson/env.sh
DT_WS=$PWD/ros2_ws DT_VENV=$DT_VENV ./sim/run_demo.sh --profile airy       #
or mid360
# options: --live (watch in Blender) --splat (photo-real) --no-hero (skip
device renders) --rate R --cam-rate HZ
python3 sim/view_twin.py   # interactive view of the simulated twin
```

Outputs go to `~/dt_data/sim/demo_<profile>/`: `result_day1.png` (twin coloured by error), `result_rgb_day1.png` (with camera colour), `result_day2.png` (changes), `eval_day1.json`, `eval_day2.json`.

Simulation results

Measure	Result
Path error (ATE)	about 1.0–1.7 cm
Map error, mean distance to true surfaces	about 2.6 cm
Surfaces coloured by the camera	87%
Gaussian splat image quality	39.1 dB PSNR

Real captures are harder than the simulation: real sensors have per-beam errors, mixed pixels at edges, auto-exposure and a person holding the rig. See document 8.

Known limits

- The simulated Airy assumes 96 evenly spaced rings over 0–90° and the IMU at the LiDAR centre.
- Camera rendering takes about 0.9 s per 720p frame on an RTX 4060, so the demo renders at 5 Hz by default.
- Use CycloneDDS with large buffers (or `--rate 0.25`) for the Airy, or FAST-LIO starves.